

DevOps → MLOps

AI Platform Engineer

18-Month Production Roadmap

PRACTICAL · END-TO-END · PRODUCTION-GRADE

PHASE	FOCUS	DURATION
1	Foundation Hardening	Months 1–3
2	Python + ML Fundamentals	Months 3–5
3	MLOps Core Stack	Months 5–9
4	LLM & Generative AI Infrastructure	Months 9–14
5	Production Platform Engineering	Months 14–18

Build in public. Every project on GitHub. Write about what you build. The compounding effect of visibility is real in this field.

Foundation Hardening

Goal: Become a rock-solid DevOps engineer first. Every later skill builds on this base.

LEARN

Linux & Networking Deep Dive

- Master systemd, cgroups, namespaces — the foundation of containers
- TCP/IP, DNS, iptables, network policies at depth
- File descriptors, sockets, process management internals
- Practice: Set up a bare-metal Ubuntu server from scratch

LEARN

Kubernetes — Production Level

- CKAD + CKA certification path (hands-on, not just theory)
- Understand etcd, kube-scheduler, kubelet internals
- Helm charts, Kustomize, GitOps with ArgoCD / Flux
- Practice: Deploy a multi-tier app on self-managed k8s (kubeadm)

LEARN

CI/CD Pipelines

- GitHub Actions / GitLab CI from scratch — build → test → push → deploy
- Secrets management with HashiCorp Vault or AWS Secrets Manager
- Practice: Build a full pipeline for a Python FastAPI application

LEARN

Observability Stack

- Prometheus + Grafana (metrics), Loki (logs), Tempo (traces)
- Write custom exporters and alerting rules
- OpenTelemetry instrumentation basics
- Practice: Monitor a real app — set up SLOs and alert on them

ROCKET

PROJECT

Phase 1 Projects to Build

- Deploy a 3-tier app on k8s with full observability stack
- Automate infrastructure with Terraform on AWS or GCP
- Set up GitOps: any push to main auto-deploys to cluster

Python + ML Fundamentals

Goal: Learn just enough ML to understand what you're deploying. You don't need to train models — you need to understand them.

LEARN

Python for DevOps / MLOps

- Python beyond scripting: classes, decorators, async/await
- FastAPI for building ML serving APIs
- Pydantic for data validation in ML pipelines
- Practice: Build a REST API that serves a simple ML model

LEARN

ML Concepts — Practical, Not Academic

- Training vs inference — understand the compute difference
- Model formats: ONNX, TorchScript, SavedModel
- Batch vs real-time inference trade-offs
- GPU basics: CUDA, VRAM, why it matters for infrastructure
- Read: 'Designing ML Systems' by Chip Huyen (must-read)

LEARN

Data Pipeline Basics

- Apache Airflow: DAGs, operators, scheduling patterns
- Pandas and data validation with Great Expectations
- S3/GCS as a data lake — partitioning strategies
- Practice: Build an Airflow DAG — ingest, validate, store in S3

ROCKET

PROJECT

Phase 2 Projects to Build

- Deploy a pre-trained HuggingFace model behind a FastAPI endpoint
- Build an Airflow pipeline: fetch data → process → store in S3
- Containerise it all with Docker, push to ECR/GCR

Goal: *This is your main differentiator. Learn to operationalise ML like production software.*

LEARN

Experiment Tracking & Model Registry

- MLflow: tracking experiments, logging metrics, model registry
- Weights & Biases (W&B;) — used heavily in industry
- Model versioning and lineage concepts
- Practice: Track 5 experiments with different params, promote best to registry

LEARN

Model Serving at Scale

- Triton Inference Server (NVIDIA) — industry standard for serving
- TorchServe and TF Serving for framework-native serving
- Seldon Core or BentoML on Kubernetes
- Latency vs throughput trade-offs, dynamic batching strategies
- Practice: Serve a model on Triton, load test with Locust

LEARN

Feature Stores

- Feast (open source) — feature views, online/offline store concepts
- Why feature stores exist: the training-serving skew problem
- Redis as an online store for low-latency feature serving
- Practice: Build a feature pipeline with Feast + Redis

LEARN

ML Pipeline Orchestration

- Kubeflow Pipelines — end-to-end ML pipelines on Kubernetes
- ZenML — modern, clean MLOps framework worth learning
- Practice: Build a full retrain pipeline: ingest → train → evaluate → deploy

LEARN**Model Monitoring**

- Data drift detection with Evidently AI
- Model performance degradation: PSI, KL divergence concepts
- Alerting on prediction distribution shifts
- Practice: Simulate data drift and alert on it in Grafana

ROCKET**PROJECT****Phase 3 Projects to Build**

- End-to-end ML pipeline: raw data → trained model → deployed API → monitored
- Auto-retrain trigger: drift detected → pipeline kicks off → new model deployed
- Model A/B testing on Kubernetes with traffic splitting

Goal: Where DevOps and AI meet at the frontier. Very few engineers know this space well yet.

LEARN

LLM Serving Infrastructure

- vLLM — production LLM serving engine (PagedAttention, continuous batching)
- Ollama for local dev, TGI (HuggingFace) for production serving
- OpenAI-compatible API layer — how to build a proxy/gateway
- Quantization: GPTQ, AWQ, GGUF — reduce VRAM while maintaining quality
- Practice: Self-host Llama 3 on GPU instance with vLLM, benchmark throughput

LEARN

GPU Infrastructure on Kubernetes

- NVIDIA GPU operators on Kubernetes
- Multi-GPU strategies: tensor parallelism, pipeline parallelism
- GPU monitoring: DCGM Exporter + Grafana dashboards
- Spot/preemptible GPU instances for cost optimisation
- Practice: Deploy a GPU node pool on GKE/EKS, run multi-GPU inference

LEARN

RAG Systems Infrastructure

- Vector databases: Qdrant, Weaviate, pgvector — production trade-offs
- Embedding pipelines — async, scalable, with retry logic
- Chunking strategies and their infrastructure implications
- Practice: Build a production RAG pipeline with ingestion + serving layer

LEARN

LLM Observability

- LLM-specific metrics: TTFT, TPS, token throughput, queue depth
- Prompt/response logging with LangSmith or Langfuse
- Cost tracking per request, per user, per model
- Guardrails infrastructure: content filtering, PII detection at scale
- Practice: Build a full observability stack for an LLM API

LEARN**AI Gateway & Platform**

- Build an internal AI gateway: routing, rate limiting, auth, fallback models
- LiteLLM proxy — route between OpenAI, Anthropic, self-hosted models
- Semantic caching with Redis + embeddings to cut costs
- Practice: Gateway routing to 3 providers with automatic fallback logic

ROCKET**PROJECT****Phase 4 Projects to Build**

- Production RAG system: doc ingestion → vector DB → LLM API → full observability
- LLM Gateway: auth + routing + rate limiting + cost dashboard
- Auto-scaling LLM serving: scale GPU pods based on queue depth

Goal: Graduate from engineer to platform owner. Build the internal developer platform for AI teams.

LEARN

Internal Developer Platform (IDP)

- Backstage.io — build a service catalog for all ML services
- Self-service infrastructure with Crossplane or Terraform Cloud
- Golden paths: templates for 'new ML service in 5 minutes'
- Practice: Build a Backstage portal with ML service templates

LEARN

Cost Engineering (FinOps for AI)

- GPU cost per inference, per model, per team — full visibility
- Spot instance strategies for training workloads
- Resource quotas and chargeback models for AI teams
- OpenCost or Kubecost for Kubernetes cost visibility
- Practice: Build a cost dashboard breaking down all AI infra spend

LEARN

Security for AI Systems

- Model supply chain security — signing and verifying model artifacts
- RBAC for model registry access
- PII and sensitive data handling in training pipelines
- Prompt injection defence at the infrastructure level
- Practice: Implement model artifact signing with Sigstore / Cosign

LEARN

Reliability & Disaster Recovery

- SLOs for ML APIs: latency p99, availability, prediction freshness
- Chaos engineering for ML: what happens when the model is slow?
- Multi-region model serving with automatic failover
- Practice: Run a GameDay — kill your LLM infra and recover in under 15 mins

Capstone Projects

- Build a complete AI Platform: onboarding → training → serving → monitoring → cost
- Write architecture decision records (ADRs) for every major design choice
- Open source one tool OR write 3 detailed technical blog posts
- Contribute to an open source MLOps project (Kubeflow, MLflow, ZenML, etc.)

CERTIFICATIONS

Signal credibility to hiring managers

CKA / CKAD	Month 2–3	Must Have
AWS / GCP Professional Cloud Architect	Month 4–5	Must Have
Terraform Associate	Month 3–4	Recommended
AWS ML Specialty	Month 10–12	Recommended
NVIDIA DLI — GPU Infrastructure	Month 10–11	Niche Edge

Strategy: Don't chase certs without hands-on projects. A GitHub repo with 3 real MLOps projects beats 5 certs with no practical work. Certs open doors — projects close offers.

RESOURCES

Curated — only what matters in production contexts

BOOKS

Designing ML Systems — Chip Huyen	Phase 2
The Phoenix Project — Gene Kim	Phase 1
Site Reliability Engineering — Google SRE Book	Phase 1

COURSES

MLOps Specialization — DeepLearning.AI (Coursera)	Phase 3
Full Stack Deep Learning (FSDL)	Phase 3–4
Made With ML — Goku Mohandas (free)	Phase 3

REPOS

mlops-zoomcamp — DataTalks.Club (GitHub)	Phase 3
--	---------

BLOGS

huyenchip.com — Chip Huyen	Ongoing
----------------------------	---------

Remember

Build in public. Every project on GitHub.

Write about what you build — LinkedIn or a blog.

The compounding effect of visibility is real in this field.

You're not starting over.

You're evolving — and that's a rare, powerful position.